PYTHON for Devops:-

Python is preferred over bash for scripting is because of reasons like:

→ Python works everywhere while bash works with Linux / Unix system only.

→ Python has very huge ecosystem for libraries.

→ Handling error is strong in python while weak in bash.

→ Bash use case : System automation
Python:- Adv. Scripting, cloud automation.

For simple automation and quicks like:

→ copying files
→ Running system commands
→ managing users or cron jobs

Bash is faster & simpler

→ many infra. automation scripts and CI/CD pipelines are written in Python for better scalability & clarity.

→ Python scripts run on any OS (Linux, mac, Windows) Bash scripts are primarily for Unix / Linux.

Python indentation:-

→ Identation refers to the spaces at the beginning of a code line.

→ python uses indentation to indicate a block of code.

→ number of spaces as a programmer is up to you but, most common use is four, but it has to be least one.

Comments:

comments starts with #, & are ignored.

Why is python dynamic in nature:-

Because in python we dont declare the type of variable Python figures it out automatically at runtime.

e.g.-
```
x = 10      # int
x = "Alok"  # string
```

Python is dynamic hence it is
- → great for scripting, automation & rapid development.
- → more flexible: can easily handle diff data types.

Multiline Strings as Comments (Docstrings):-

We can use (''' or """) to write multiline text,

→ These are technically multi-line strings but if they're not assigned to any variable or used as a docstring, Python ignores them — so they can act like multi-line comments.

e.g.-
```
""" 
This is multiline string,
which acts as multiline comment.
"""
```

* Its not a true comment, just an ignored string. *

Data Types :-

Before understanding data types, we have to understand ~~data~~ variables.

variables :-

variables are containers for storing data values.

e.g:-
```
x = 5
x = "alok"
print (x)
```

output
```
alok
```

Here variable x is reassigned, and now it points towards string not integer

In python, variable are not bound to a single data-type they can change at runtime.

→ we can get the data type of variable with type() function.

- String variables can be declared either by using ' ' or " " (single quotes or double quotes)

```
a = "alok"
print (a)
a = "dev"
print (a)
```

output
```
alok
dev
```

*
```
a = 'abhay'   # we can write strings in single quote also.
print (a)
```

output
```
abhay
```

Case-sensitive

variable names are case sensitive.

```
a = 4
A = 5
print(a, A)
```

Output

4 5

way to declare multiple variables of diff. data types in one line

```
x, y, z = 1, 2.5, "Alok"
```

Variable names:

a variable can have short name like age, car_name etc.

rules for writing python variables:-

→ name must start with a letter or underscore.
→ name cannot start with number
→ name cannot be of any python keywords.
→ name can only contain alpha numeric char. & underscores. (A-Z, 0-9, _)
→ variable name are case-sensitive (age, Age, AGE are 3 diff. variables)

Legal names:-

myvar
my_var
_myvar
MYVAR
myvar2

illegal variables

2myvar
my-var
my var

Multiwords variable names:-

variable names with more than one word can be difficult to read.
There are several techniques we can use to make them more readable.

camelcase → myVariableName
Pascalcase → MyVariableName
snake case → my_variable_name

assigning diff. values to ~~many~~ multiple variables:

x, y, z = "orange", "apple", "banana"

assigning single value to multiple variables:-

x = y = z = "orange"

global variable:- variable created outside of a function
→ global variable can be used both inside & outside of function.

Note:- if you create a variable with the same name inside a function, this variable will be local & can only be used inside the function.

global keyword: when we create a variable inside a function, it is local variable, but if we use global keyword we can create a global variable inside a function.

e.g:

```
x = "awesome"        # global variable
def myfunc():
    global x         # makes x global variable
    x = "fantastic"  # changes its value
    print("Python is", x)
myfunc()
print("Python is", x)
```

Output

```
Python is fantastic
Python is fantastic
```

→ basically it changes the global variable value from awesome to fantastic which was declared outside the function.

```
x = "awesome"        # global variable
def myfunc():
    x = "fantastic"  # local variable
    print("Python is", x)
myfunc()
print("Python is", x)
```

Output

```
Python is fantastic
Python is awesome.
```

Data Types in python:

variables can store data of diff. types, & diff. types of data do diff. things.

Text type : str
Numeric types : int, float, complex
Sequence type : list, tuple
mapping type : dict
Set types : sets, frozenset
Boolean types : bool
None type : NoneType

Numeric data type :-

→ int
→ float
→ complex

e.g.
```
x = 1    # int
y = 2.8  # float
z = 1j   # complex.

print (type (x))
print (type (y))
print ( type (z))
```

Type Conversion :-

```
x = 1
y = 2.8
z = 1j
a = float (x)
b = int (y)
c = complex (x)
X d = int (z)
print (type (a))
print (type (b))
print (type (c))
X print ( type (d))
```

we can convert
int → float, complex
float → int, complex

Note: we cannot convert complex no. into another numeric type.

Random Number:-

Python does not have a random() function to make a random number, but it has a built in module called random which is used to make random numbers.

```
import random
print ( random.randrange (1, 10))
```

Strings:-

→ Strings are surrounded by either single quotation marks or double quotation marks.

'hello' is the same as "hello"

→ We can display a string literal using print function

```
print ("Hello")
```

assign string to a variable:

```
a = "Hello"
print (a)
```

multiline strings:

```
a = """ Hello everyone,
This is a,
multiline string """
print (a)
```

Strings are arrays :-

```
a = "Hello, World"
print (a[1])
```

Hello, World
0 1 2 3 4 5 6 7 8 9 10 11

Output

e

→ Since strings are arrays, we can loop through char in a string using for loop.

eg:-
```
for x in "banana":
    print (x)
```

b
a
n
a
n
a

→ To get the length of a string use len() function.

```
print (len (a))
» 12
```

Check strings :

To check if a certain phrase or character is present in a string, we can use in keywords.

eg:
```
txt = "The best things in life are free"
print ("free" in txt)
» True          # if free is not present it will return false.

txt = "Hello, How are you"
if "are" in txt:
    print ("'are' is present")
```

→ if are is present in txt it will print 'are' is present and if not this will print nothing.

```
txt = "Avengers is a marvel movie"
print ("marvel" not in txt)
```

» False

Slicing :-

return range of characters in a string.

```
b = "Hello World"
print (b[1:4])
```

↳ it is not included

Output

```
ell
```

prints the character from index no 1 to 3, 4th char. is not included.

→ `print (b[:5])` (prints char. from starting index to index 4)

» Hello

→ `print (b[2:])` (slice to end)

» llo World

Negative indexing :-

```
→ print (b[-5:-2])
```

» Wor

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

seq[a:b] → from a to b-1
seq[a:] → from a to end
seq[:b] → from start to b-1
seq[:] → full sequence
seq[::-1] → reversed sequence.

→ print(b[::-1])

» dlroW olleH

format

seq.[start: End: step]

→ print(b[::2])

» HloWrd

↳ prints every 2nd char.

H e l l o W o r l d
0 1 2 3 4 5 6 7 8 9 10

→ print(b[4:11:3])

» o od ↳ prints every 3rd char. from ind. 4 to 11 -1

Modify strings:-

Python has set of built in methods that you can use on strings.

→ **upper case :-**

```
a = "Hello World"
print(a.upper())
```

» HELLO WORLD

→ **Lower case**

```
print(a.lower())
```

» hello world

→ **Remove whitespaces**

```
print(a.strip())
```

» Hello World

→ **Replaces string**

```
print(a.replace("Hello", "Alok"))
```

» Alok World

Split string :

split() method divides a string into a list of words (or parts), based on separator (delimiter).

→ print (a.split(','))
» ['Hello, World']

→ print (a.split())
» ['Hello', 'World']

→ a = "Hello, World"
print (a.split(','))
» ['Hello', 'World']

String Concatenation :

combine two strings or two or more strings

```
a = "Python"
b = "is"
c = "great"
result = a + " " + b + " " + c
print(result)
```

» Python is awesome

As we know that, we cannot combine strings & numbers like this :

But we can combine strings & number by using f-strings or format method.

f - string :-

To specify a string as f-string put f in front of string literal and curly brackets {} as placeholders for variables.

eg:

```
age = 36
txt = f"my name is Alok, age is {age}"
print(txt)
```

>> my name is Alok, age is 36

eg:

```
txt = f"Price of laptop is {1000 * 1000} rupees"
print(txt)
```

>> Price of laptop is 1000000 rupees

Escape character :-

To insert characters that are illegal in string, use an escape char.

→ An escape char is \.

eg: We can not use a double quote inside a double quote string.

→

```
text = "We are \"Vikings\" from north."
print(text).
```

>> We are "Vikings" from north.

Boolean :-

Boolean represents two possible values :

→ True or False.

e.g. → print (10 > 9)

» True

→ print (10 == 9)

» False

e.g:

```
a = 100
b = 33
if b > a:
    print ( "b is greater than a")
else :
    print ( " b is not greater than a")
```

» b is not greater than a

prints a message based on condition whether it is true or false.

bool() function:

built-in function that converts any value into a boolean value.

syntax - bool (value)

If you didn't pass any value, it returns false

e.g:- 1) for numbers :-

rules : Any non-zero number = True

~~for~~ zero 0 → false.

→ print (bool (10)) >> True
→ print (bool ()) >> False.

2) For strings:-

Non-empty strings → True
Empty string → False

e.g.
→ print (bool ("Hello")) → True
→ print (bool (" ")) → False

3) For lists, tuples, dictionaries:

e.g. → print (bool ([1, 2, 3])) >> True. } Lists
→ print (bool ([])) >> False

→ print (bool (())) >> False } Tuple
→ print (bool ((2))) >> True

→ print (bool ({})) >> False } Dictionary
→ print (bool ({"name" : "alok"})) >> True

4) For None

→ print (bool (None)) >> False

True = 1
False = 0

→ print (True + True)
>> 2

print (False + False)
>> 0

print (True + False)
>> 1

Function :-

→ A function is a block of code which only runs when it is called.

→ In python a function is defined using def keyword.

eg:- def my_function():
    print("Hello from function")

Now call the function with its name followed by parenthesis

```
my_function()
# this will print
>> Hello from function
```

Function with parameter :-

```
def greet(name)
    print("Hello", name)

greet("Alok")
>> Hello Alok
```

Here "Alok" is an argument (actual value passed)
& name is a parameter (variable inside the function)

Function with multiple value :-

```
def add(a, b):
    print("the sum is", a+b)

add(4, 5)
>> 9  #output
```

Function with return value :-

Function can also return a value using return

e.g:-

```
def add(a, b):
    return a + b
result = add(5, 3)
print(result)
```

Function with default parameter value :-

```
def greet(name = "Alok")
    print("Hello", name)

greet()
greet("Abhay")
```

» Hello Alok
» Hello Abhay

∴ In this function if we didn't pass any value while calling function, it will return the default value, & if we pass other value except the default it will return, the value we pass while calling function

Python Modules :-

Module is simply a python file that contain functions, variables & classes → all grouped together so we can reuse them in other programs.

Main advantage of using modules is reusability

function with return :

```
def addition (num1, num2)
        add = num1 + num2
        return add

print (addition ( 2, 3))
```

>> 5

For eg.

```
import math   # math is a built-in module
print(math.sqrt(16))   # 4.0
print (math.pi)  # 3.141592653589793

# random module
import random
print (random.randint(1,10))  # prints random no
                                btw 1 & 10

# datetime module.
import datetime
x = datetime.datetime.now()
print(x)

# os module
import os
print (os.name)          # system name
print (os.getcwd())  # prints current working
                       directory.
```

If we want to import specific things :

```
from math import sqrt, pi

print (sqrt(25))
print (pi)
```

By this we can directly use sqrt() instead of math.sqrt()

Rename a module:-

```
import math as m
print(m.sqrt(9))
```

In simple term:
A module can include
→ functions
→ variables / constants
→ classes
→ other modules via import. (It uses other modules, but it doesn't contain them)

Packages :-

A package is a collection of Python modules. organized in directories (folders)

Structure of a package :-

A package is a folder with special file called
__init__.py → this file can be empty or it can contain initialization code.

e.g. my_package/
→ __init__.py
→ module1.py
→ module2.py
sub_package/
→ __init__.py
→ module3.py

Importing modules from package

```
from my-package import module1
module1.func()
```

or import specific function

```
from my-package.module1 import func
func()
```

Nested package e.g.:

→ `from my-package.sub-package import module3`
`module3.some-function()`

→ `from my-package.sub-package.module3 import func`
`func()`

Virtual Environment (venv)

→ It is self contained python environment.

→ It allows you to install packages separately for each project (acc to their requirement & required version) without affecting your system wide python installation

∴ avoids conflicts between package versions across projects.

Why use virtual environments:-

→ Project isolation: diff. projects use diff. version of the same package.

→ Avoid conflicts:- Prevents breaking other projects.

→ Clean workspace: Only packages needeed for the project are installed.

1) Creating a virtual environment:

python -m venv myenv

→ myenv named folder created in your project directory.

2) Activating the virtual environment:-

On windows

.\myenv\scripts\activate

.\ → means run from the current directory.

On macos / Linux

source myenv/bin/bash activate

→ After activation, you will see (myenv) at the beginning of your terminal prompt.

3) Install Packages

Once virtual environment is active, use pip to install packages locally in the environment.

pip install requests

4) Deactivating virtual environment:-

deactivate.

→ Your terminal will return to the global environment.

5) Sharing the ~~requirements~~:- Environment:

create a requirements.txt file to share dependencies:

pip freeze > requirements.txt

Command Line Arguments:- (uses sys)

Command Line Arguments are values you pass to a Python script when you run it from the terminal (command line).

→ It can behave differently based on user input without changing the code.

e.g. We have a python file named greet.py

```
import sys
print ("Arguments:", sys.argv)
```

When we run this file in terminal like this:

» python greet.py Alok Patel

Output

→ Arguments: ['greet.py', 'Alok', 'Patel']

Here:

→ sys.argv is a list in python that stores all command line arguments.

→ The first element from that list (sys.argv[0]) is always the name of the script.

→ Next elements sys.argv[1], sys.argv[2] are actual arguments you passed.

e.g.:
```
import sys
name = sys.argv[1]
print ("Hello,", name)
```

» python greet.py Alok

Output

Hello, Alok

Print two numbers

```
import sys
num1 = int(sys.argv[1])
num2 = int(sys.argv[2])

sum = num1 + num2
print("sum:", sum)
```

```
» python test.py 10 20
» 30
```

Environment variables :- (env vars) (uses OS)

→ Environment variables are key-value pairs stored outside your python program, usually in your OS.

→ They are used to store configuration or sensitive information (like passwords, API keys or database URLs) that your program can access at runtime — without hardcoding them inside your code.

How to set environment variables:-

1) On windows (Command Prompt):

```
set my_name = "alok"
python app.py
```

2) On linux / mac :

```
export my_name = "alok"
python3 app.py
```

3) On powershell

```
$env: my_passwd = "alok"
```

Then in your python file :-

```
import os
print(os.getenv("my_name"))
```

List all environment variable

```
import os
for key, value in os.environ.items():
    print(f"{key} = {value}")
```

E.g. Using Environment variables for secrets :

```
import os
api_key = os.getenv("API_KEY")
if api_key:
    print("API Key found")
else:
    print("API Key not found. Please set it.")
```

Then set API key in terminal:

```
$env: API_KEY = 1234abcd.
```

then run python file

```
python app.py
```

We can also use environment variable by .env file :

→ create .env file in same folder as your main python script. & store or add variables inside .env file.

```
my_passwd = alok
API_KEY = 1234abcd
```

Install python-dotenv library

↳ this library read variables from your .env file automatically.

```
pip install python-dotenv
```

→ then load .env file in your Python Script :-

```
from dotenv import load_dotenv
import os

load_dotenv()
print(os.getenv("my_passwd"))
print(os.getenv("API_KEY"))
```

then run python script

```
python app.py
```

```
» alok
» 1234abcd
```

Python Operators :

Operators are used to perform operations on variables & values.

1) Arithmatic operator :-

+ →	addition	10 + 5 → 15
− →	subtraction	10 − 5 → 5
* →	multiplication	10 * 5 → 50
/ →	Division (gives result in point)	10/3 → 3.333
// →	Floor division (gives integer result)	10//3 → 3
% →	modulus (gives remainder)	10 % 3 → 1
** →	Exponentiation (power)	2 ** 3 → 8

2) Assignment operators :-

=	a = 5	
+=	a = a + 5	or a += 5
-=	a = a − 2	or a -= 2
*=	a = a * 2	or a *= 2
/=	a = a / 2	or a /= 2